

第4章 数据结构

4.1 基础数据结构

本节介绍与基础数据结构，包括线性表（栈、队列、链表）、二叉树、图、并查集相关的经典题目以及代码实现。尽管这些内容本身并不算“高级”，但却是很多高级内容的基础。如果数据结构基础没有打好，很难设计出正确、高效的算法。

例4-1 【优先级队列】阿格斯（Argus，北京2004, POJ 2051）

你的任务是编写一个名称为Argus的系统。该系统支持如下Register命令：

```
Register Q_num Period
```

该命令注册了一个触发器，它每Period秒钟就会产生一次编号为Q_num的事件。你的任务是模拟出前 k 个事件，其中 $1 \leq Q_num$ ， $Period \leq 3000$ ， $k \leq 10\ 000$ 。如果多个事件同时发生，先处理Q_num小的事件。输出 k 行，即前 k 个事件的Q_num。

【代码实现】

```

// 陈锋
#include <cstdio>
#include <queue>
using namespace std;

struct Item {                                // 优先队列中的元素
    int QNum, Period, Time;
    // 重要! 优先级比较函数, 优先级高的先出队
    bool operator<(const Item& a) const {    // const 修饰符不可少
        if (Time != a.Time) return Time > a.Time;
        return QNum > a.QNum;
    }
};

int main() {
    priority_queue<Item> pq;
    char s[20];
    for (Item it; scanf("%s", s) && s[0] != '#'; pq.push(it)) {
        scanf("%d%d", &it.QNum, &it.Period);
        it.Time = it.Period;                // 初始化“下一次事件的时间”为它的周期
    }
    int K;
    scanf("%d", &K);
    while (K--) {
        Item r = pq.top();                  // 取下一个事件
        pq.pop();
        printf("%d\n", r.QNum);
        r.Time += r.Period;                 // 更新该触发器的“下一个事件”的时间
        pq.push(r);                         // 重新插入优先队列
    }
    return 0;
}
/*
算法分析请参考:《算法竞赛入门经典——训练指南》升级版 3.1.2 节例题 3
同类题目: Queue and A, ACM/ICPC WF 2000, UVa 822
*/

```

例4-2 【栈的使用】铁轨 (Rails, ACM/ICPC CERC 1997, POJ 1363)

某城市有一个火车站, 铁轨铺设如图4-1所示。有 n 节车厢从 A 方向驶入车站, 按进站顺序编号为 $1 \sim n$ 。你的任务是判断是否能让它们按照某种特定的顺序进入 B 方向的铁轨并驶出车站。例如, 出栈顺序(5, 4, 1, 2, 3)是不可能的, 但(5, 4, 3, 2, 1)是可能的。

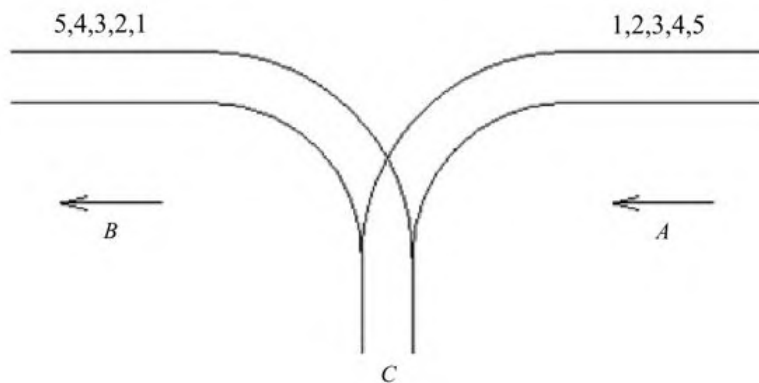


图4-1 铁轨

为了重组车厢，你可以借助中转站 C 。这是一个可以停放任意多节车厢的车站，但由于末端封顶，驶入 C 的车厢必须按照相反的顺序驶出 C 。对于每节车厢：一旦从 A 移入 C ，就不能再回到 A ；一旦从 C 移入 B ，就不能再回到 C 。换句话说，在任意时刻，只有两种选择： $A \rightarrow C$ 和 $C \rightarrow B$ 。

【代码实现】

```

// 陈锋
#include <stack>
#include <iostream>

using namespace std;
#define _for(i, a, b) for (int i = (a); i < (b); ++i)
#define _rep(i, a, b) for (int i = (a); i <= (b); ++i)
const int MAXN = 1000 + 4;
int N, B[MAXN];
int main() {
    ios::sync_with_stdio(false), cin.tie(0);
    while (cin >> N && N) {
        while (cin >> B[1] && B[1]) {
            _rep(i, 2, N) cin >> B[i];
            stack<int> s;
            int ai = 1, bi = 1;
            while (bi <= N) {
                if (ai == B[bi]) ai++, bi++; // 从 A->C->B, 不用入栈
                else if (!s.empty() && s.top() == B[bi]) ++bi, s.pop(); // 栈顶的开过去
                else if (ai <= N) s.push(ai++); // 只能先入栈试试看
                else break; // 无计可施了
            }
            cout << ((bi == N + 1) ? "Yes" : "No") << endl;
        }
        cout << endl;
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典（第2版）》例题 6-2
注意：在获取栈顶元素之前要判断栈是否为空
同类题目："Accordian" Patience, UVa 127
        删除物品, JLOI 2013, NC 20134
*/

```

例4-3 【双端队列】并行程序模拟（Concurrency Simulator, ACM/ICPC WF1991, UVa 210）

你的任务是模拟 n 个程序（按输入顺序编号为 $1 \sim n$ ）的并行执行。每个程序包含不超过25条语句，格式一共有5种：
 var=constant（赋值）；print var（打印）；lock；unlock；
 end。

变量用单个小写字母表示，初始为0，为所有程序公有（在一个程序里对某个变量赋值可能会影响另一个程序）。常数是小于100的非负整数。

每个时刻只能有一个程序处于运行态，其他程序均处于等待态。上述5种语句分别需要 t_1, t_2, t_3, t_4, t_5 单位时间。运行态的程序每次最多运行 Q 个单位时间（称为配额）。当一个程序的配额用完之后，把当前语句（如果存在）执行完之后该程序会被插入一个等待队列中，然后处理器从队首取出一个程序继续执行。初始等待队列包含按输入顺序排列的各个程序，但由于lock/unlock语句的出现，这个顺序可能会改变。

lock的作用是申请对所有变量的独占访问。lock和unlock总是成对出现，并且不会嵌套。lock总是在unlock的前面。当一个程序成功执行完lock指令之后，其他程序一旦试图执行lock指令，就会马上被放到一个所谓的阻止队列的尾部（没有用完的配额就浪费了）。当unlock执行完毕后，阻止队列的第一个程序进入等待队列的首部。

输入 $n, t_1, t_2, t_3, t_4, t_5, Q$ 以及 n 个程序，按照时间顺序输出所有print语句的程序编号和结果。

【代码实现】

```
// 陈锋
#include <bits/stdc++.h>
#define _for(i,a,b) for( int i=(a); i<(int)(b); ++i)
using namespace std;
const int NN = 1000 + 4;
```

```

deque<int> readyQ;
queue<int> blockQ;
int N, Quantum, C[5], Var[26], IP[NN];    // IP[pid]是程序 pid 运行的当前行号
bool locked;
string Stats[NN];                        // 程序语句

void run(int pid) {
    int q = Quantum;
    while (q > 0) {
        const string& p = Stats[IP[pid]];
        switch (p[2]) {
            case '=':
                Var[p[0] - 'a'] = isdigit(p[5]) ? (p[4] - '0') * 10 + p[5] - '0' : p[4] - '0';
                q -= C[0];
                break;
            case 'i':
                // print
                cout << pid + 1 << ": " << Var[p[6] - 'a'] << endl;
                q -= C[1];
                break;
            case 'c':
                // lock
                if (locked) { blockQ.push(pid); return; }
                locked = true;
                q -= C[2];
                break;
            case 'l':
                // unlock
                locked = false;
                if (!blockQ.empty()) {
                    int pid2 = blockQ.front(); blockQ.pop();
                    readyQ.push_front(pid2);
                }
                q -= C[3];
                break;
            case 'd':
                // end
                return;
        }
        IP[pid]++;
    }
    readyQ.push_back(pid);
}

int main() {
    ios::sync_with_stdio(false), cin.tie(0);
    int T; cin >> T;
    while (T--) {
        cin >> N;
        _for(i, 0, 5) cin >> C[i];
        cin >> Quantum;
        fill_n(Var, 26, 0);
        int line = 0;
    }
}

```

```

_for(i, 0, N) {
    getline(cin, Stats[line++]);
    IP[i] = line - 1;
    while (Stats[line - 1][2] != 'd') getline(cin, Stats[line++]);
    readyQ.push_back(i);
}

locked = false;
while (!readyQ.empty()) {
    int pid = readyQ.front(); readyQ.pop_front();
    run(pid);
}
if (T) cout << endl;
}
return 0;
}
/*
算法分析请参考：《算法竞赛入门经典（第2版）》例题 6-1
同类题目：10-20-30, ACM/ICPC WF 1996, UVa 246
*/

```

例4-4 【从中序和后序恢复二叉树】树 (Tree, UVa 548)

给出一棵点带权（权值各不相同，都是小于10 000的正整数）的二叉树的中序和后序遍历，寻找一片叶子，使得它到根的路径上的权值和最小。如果有多个解，该叶子本身的权值应尽量小。输入中，每两行表示一棵树，其中第一行为中序遍历，第二行为后序遍历。

【样例输入】

```

3 2 1 4 5 7 6
3 1 2 5 6 7 4
7 8 11 3 5 16 12 18
8 3 11 7 16 18 12 5
255
255

```

【样例输出】

```

1
3
255

```

【代码实现】

```
// 陈锋
#include <bits/stdc++.h>
using namespace std;
typedef long long LL;
struct Node {
    Node *left, *right;
    int val;
    Node() : left(nullptr), right(nullptr) {}
```



```

~Node() {
    if (left) delete left;
    if (right) delete right;
    left = right = nullptr;
}
};

const int MAXN = 10000 + 4;
int In[MAXN], Post[MAXN], N;

int parse(const string &str, int *p) {
    stringstream ss(str);
    int n = 0, x;
    while (ss >> x) p[n++] = x;
    return n;
}

Node *parseTree(int il, int i2, int pl, int p2) { // In[il, i2], Post[pl, p2]
    assert(il <= i2 && pl <= p2);
    Node *p = new Node();
    int m = Post[p2];
    p->val = m;
    if (il == i2) {
        assert(pl == p2 && In[il] == Post[p2]);
        return p;
    }
    int ri = find(In + il, In + i2 + 1, m) - In,
        lLen = ri - il; // root, left tree
    if (il <= ri - 1) p->left = parseTree(il, ri - 1, pl, pl + lLen - 1);
    if (ri + 1 <= i2) p->right = parseTree(ri + 1, i2, pl + lLen, p2 - 1);
    return p;
}

ostream &operator<<(ostream &os, Node *pn) {
    if (pn->left) os << "(" << pn->left << ")";
    os << pn->val;
    if (pn->right) os << "(" << pn->right << ")";
    return os;
}

void dfs(int sum, Node *p, int &minSum, int &minLeaf) {
    assert(p);
    if (p->left || p->right) {
        if (p->left && sum + p->left->val <= minSum)
            dfs(sum + p->left->val, p->left, minSum, minLeaf);
        if (p->right && sum + p->right->val <= minSum)
            dfs(sum + p->right->val, p->right, minSum, minLeaf);
    } else {
        if (sum < minSum)
            minSum = sum, minLeaf = p->val;
        else if (sum == minSum)

```

```

        minLeaf = min(p->val, minLeaf);
    }
}

int main() {
    string l1, l2;
    while (getline(cin, l1) && getline(cin, l2)) {
        N = parse(l1, In);
        assert(N == parse(l2, Post));
        Node *pTree = parseTree(0, N - 1, 0, N - 1);
        int minSum = INT_MAX, minLeaf = INT_MAX;
        dfs(pTree->val, pTree, minSum, minLeaf);
        delete pTree;
        cout << minLeaf << endl;
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典（第2版）》例题 6-8
同类题目：Tree Recovery, ULM 1997, UVa 536
*/

```

例4-5 【二叉树的DFS】下落的树叶（The Falling Leaves, UVa 699）

给出一棵二叉树，每个结点都有一个水平位置：左子结点在它左边1个单位，右子结点在它右边1个单位。从左向右输出每个水平位置的所有结点的权值之和。如图4-2所示，从左到右的3个水平位置的权值和分别为7，11，3。按照递归（先序）方式输入，用-1表示空树。

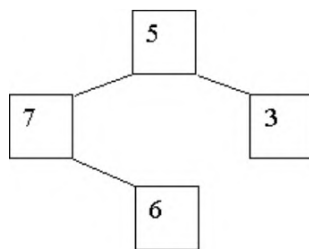


图4-2 结点权值

【样例输入】

```
5 7 -1 6 -1 -1 3 -1 -1
```

```
8 2 9 -1 -1 6 5 -1 -1 12 -1 -1 3 7 -1 -1 -1
-1
```

【样例输出】

Case 1:

```
7 11 3
```

Case 2:

```
9 7 21 15
```

【代码实现】

```
// 陈锋
#include <bits/stdc++.h>
using namespace std;
typedef long long LL;
#define _for(i, a, b) for (int i = (a); i < (b); ++i)
map<int, int> sums;
void readTree(int pos) {
    int root;
    scanf("%d", &root);
    if (root == -1) return;
    sums[pos] += root, readTree(pos - 1), readTree(pos + 1);
}

int main() {
    for (int t = 1; true; t++) {
        sums.clear(), readTree(0);
        if (sums.empty()) break;
        printf("Case %d:\n", t);
        int i = 0;
        for (auto p : sums) {
            if (i++) printf(" ");
            printf("%d", p.second);
        }
        puts("\n");
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典（第2版）》例题 6-10
注意：本题直接利用了输入的递归性质进行直接处理，无须构建内存中对应的树结构
*/
```

例4-6 【二叉树隐式DFS】天平 (Not so Mobile, UVa 839)

输入一个树状天平，根据力矩相等原则，判断其是否平衡。如图4-3所示，所谓力矩相等，就是 $W_l D_l = W_r D_r$ ，其中 W_l 和 W_r 分别为左右两边“砝码”的重量， D_l 和 D_r 分别为左右两边“砝码”到天平支点的距离。

采用递归（先序）方式输入：每个天平的格式为 W_l, D_l, W_r, D_r ，当 W_l 或 W_r 为0时，表示该“砝码”实际是一个子天平，接下来会描述这个子天平。当 $W_l = W_r = 0$ 时，会先描述左子天平，然后是右子天平。

【样例输入】

```
1
0 2 0 4
0 3 0 1
1 1 1 1
2 4 4 2
1 6 3 2
```

正确输出为YES，对应图4-4。

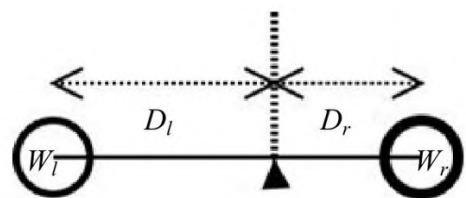


图4-3 力矩相等原则

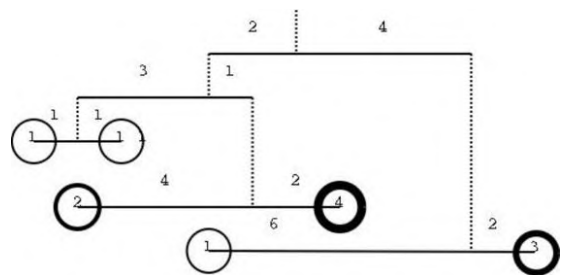


图4-4 树状天平

【代码实现】

```
// 陈锋
#include <bits/stdc++.h>
using namespace std;
typedef long long LL;
bool readTree(LL& w) {
    LL wl, dl, wr, dr;
    bool bl = true, br = true;
    cin >> wl >> dl >> wr >> dr;
    if (wl == 0) bl = readTree(wl);
    if (wr == 0) br = readTree(wr);
    w = wl + wr;
    return bl && br && (wl * dl == wr * dr);
}

int main() {
    ios::sync_with_stdio(false), cin.tie(0);
    int T;
    cin >> T;
    for (int t = 0; t < T; ++t) {
        if (t) cout << endl;
        LL w;
        bool ans = readTree(w);
        cout << (ans ? "YES" : "NO") << endl;
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典（第2版）》例题 6-9
*/
```

例4-7 【四分树】四分树 (Quadrees, UVa 297)

如图4-5所示，可以用四分树来表示一个黑白图像，方法是用根结点表示整幅图像，然后把行、列各分成两等份，按照图中的方式编号，从左到右对应4个子结点。如果某子结点对应的区域全黑或者全白，则直接用一个黑结点或者白结点表示；如果既有黑又有白，则用一个灰结点表示，并为这个区域递归建树。

给出两棵四分树的先序遍历，求二者合并之后（黑色部分合并）黑色像素的个数。p表示中间结点，f表示黑色（full），e表示白色（empty）。

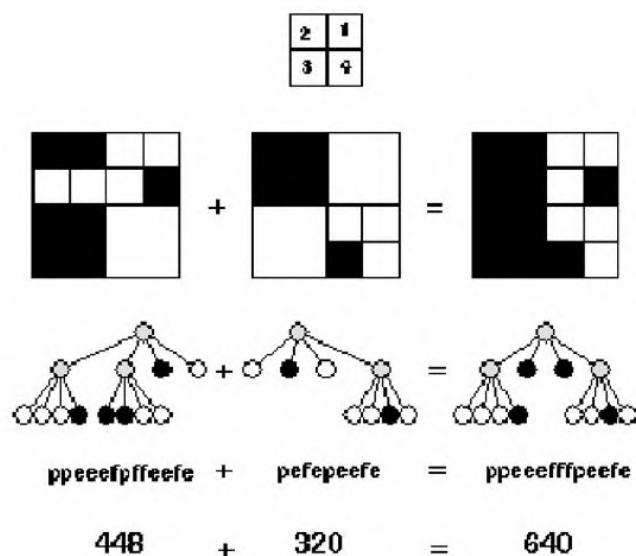


图4-5 四分树

【样例输入】

```
3
ppeeefpffeefe
pefepeefe
peeef
peefe
peeef
peepefe
```

【样例输出】

```
There are 640 black pixels.
There are 512 black pixels.
There are 384 black pixels.
```

【代码实现】

```

// 陈锋
#include <stdio>
#include <cstring>
#define _for(i, a, b) for (int i = (a); i < (b); ++i)
#define _rep(i, a, b) for (int i = (a); i <= (b); ++i)

const int len = 32, NN = 1024 + 4;
char s[NN];
int buf[len][len], cnt, DR[] = {0, 0, 1, 1}, DC[] = {1, 0, 0, 1};

// 把s[p..]画到以(r,c)为左上角, 边长为w的缓冲区中
// 2 1
// 3 4
void draw(const char* s, int& p, int r, int c, int w) {
    char ch = s[p++];
    if (ch == 'p') {
        w /= 2;
        _for(i, 0, 4) draw(s, p, r + DR[i] * w, c + DC[i] * w, w);
    } else if (ch == 'f') { // 画黑像素 (白像素不画)
        _for(i, r, r + w) _for(j, c, c + w) if (buf[i][j] == 0)
            buf[i][j] = 1, cnt++;
    }
}

int main() {
    int T;
    scanf("%d", &T);
    while (T--) {
        memset(buf, 0, sizeof(buf));
        cnt = 0;
        for (int i = 0; i < 2; i++) {
            scanf("%s", s);
            int p = 0;
            draw(s, p, 0, 0, len);
        }
        printf("There are %d black pixels.\n", cnt);
    }
    return 0;
}
/*
算法分析请参考:《算法竞赛入门经典(第2版)》例题 6-11
同类题目: Spatial Structures, ACM/ICPC WF 1998, UVa 806
*/

```

例 4-8 【使用栈进行表达式计算】矩阵链乘 (Matrix Chain Multiplication, UVa 442)

输入 n 个矩阵的维度和一些矩阵链乘表达式，输出乘法的次数。如果乘法无法进行，输出error。假定 A 是 $m \times n$ 矩阵， B 是 $n \times p$ 矩阵，那么 AB 是 $m \times p$ 矩阵，乘法次数为 $m \times n \times p$ 。如果 A 的列数不等于 B 的行数，则乘法无法进行。

例如， A 是 50×10 的， B 是 10×20 的， C 是 20×5 的，则 $(A(BC))$ 的乘法次数为 $10 \times 20 \times 5$ （矩阵 B 与矩阵 C 相乘的乘法次数）+ $50 \times 10 \times 5$ （矩阵 A 与矩阵 BC 相乘的乘法次数）=3500。

【代码实现】


```

// 刘汝佳
#include <bits/stdc++.h>
using namespace std;

struct Matrix {
    int a, b;
    Matrix(int a = 0, int b = 0): a(a), b(b) {}
} m[32];

int solve(const string& expr) {
    stack<Matrix> s;
    int ans = 0;
    for (size_t i = 0; i < expr.length(); i++) {
        char e = expr[i];
        if (isalpha(e)) s.push(m[e - 'A']);
        else if (e == ')') {
            Matrix m2 = s.top(); s.pop();
            Matrix m1 = s.top(); s.pop();
            if (m1.b != m2.a) return -1;
            ans += m1.a * m1.b * m2.b;
            s.push(Matrix(m1.a, m2.b));
        }
    }
    return ans;
}

int main() {
    int n;
    cin >> n;
    string s;
    for (int i = 0; i < n; i++) {
        cin >> s;
        cin >> m[s[0] - 'A'].a >> m[s[0] - 'A'].b;
    }
    while (cin >> s) {
        int ans = solve(s);
        if (ans == -1) printf("error\n");
        else printf("%d\n", ans);
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典（第2版）》例题 6-3
注意：Matrix 类中对于构造函数参数默认值以及成员快速初始化语法的使用
同类题目：Parentheses Balance, UVA 673
*/

```

例 4-9 【树的 BFS 与 DFS 序列转换】树重建 (Tree Reconstruction, UVa 10410)

输入一个包含 n ($n \leq 1000$) 个结点的树 BFS 序列和 DFS 序列，你的任务是输出每个结点的子结点列表。输入序列（不管是 BFS 还是 DFS）是这样生成的：当一个结点被扩展时，其所有子结点应该按照编号从小到大的顺序依次访问。

例如，若 BFS 序列为 {4, 3, 5, 1, 2, 8, 7, 6}，DFS 序列为 {4, 3, 1, 7, 2, 6, 5, 8}，则一棵满足条件的树如图 4-6 所示。

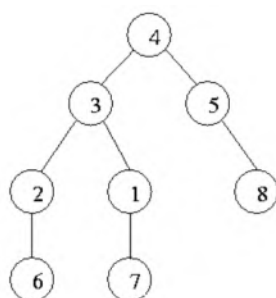


图4-6 树重建

【代码实现】

```

// 陈锋
#include <bits/stdc++.h>
using namespace std;
#define _for(i, a, b) for (int i = (a); i < (b); ++i)
#define _rep(i, a, b) for (int i = (a); i <= (b); ++i)
typedef vector<int> IVec;
typedef long long LL;
const int NN = 1004;
int N, B[NN], D[NN], BIdx[NN];
IVec G[NN];
void solve(int l, int r) { // 递归求解[l,r]子区间
    int u = D[l], i = l + 1, lasti = i;
    assert(l <= r);
    if (l == r) return;
    G[u].push_back(D[i++]); // u 的儿子
    while (i <= r) {
        int lastv = G[u].back(), v = D[i];
        if (v > lastv && BIdx[lastv] + 1 == BIdx[v])
            solve(lasti, i - 1), G[u].push_back(v), lasti = i;
        ++i;
    }
    solve(lasti, i - 1);
}
int main() {
    while (scanf("%d", &N) == 1) {
        _rep(i, 1, N) scanf("%d", &(B[i])), BIdx[B[i]] = i, G[i].clear();
        _rep(i, 1, N) scanf("%d", &(D[i]));
        solve(1, N);
        _rep(i, 1, N) {
            printf("%d:", i);
            for (size_t vi = 0; vi < G[i].size(); vi++)
                printf(" %d", G[i][vi]);
            puts("");
        }
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——习题与解答》习题 6-11
*/

```

例4-10 【优先级队列与K路合并】K个最小和（K Smallest Sums, UVa 11997）

有 k 个整数数组，分别包含 k ($1 \leq k \leq 750$) 个不超过 10^6 的正整数。在每个数组中取一个元素加起来，可以得到 k^k 个和。求这些和中最小的 k 个值（重复的值算多次）。

【代码实现】

```

// 刘汝佳
#include <bits/stdc++.h>
using namespace std;
#define _for(i, a, b) for (int i = (a); i < (b); ++i)
typedef long long LL;
const int MAXK = 768, INF = 1e6 + 4;
int K, A[MAXK], B[MAXK];
struct Item {
    int sum, b; // A[a] + B[b], b
    Item(int _sum, int _b) : sum(_sum), b(_b) {}
    bool operator<(const Item& i) const { return sum > i.sum; };
};

void merge() { // AxB -> A
    priority_queue<Item> Q;
    _for(i, 0, K) Q.push(Item(A[i] + B[0], 0));
    _for(i, 0, K) {
        Item it = Q.top();
        Q.pop(), A[i] = it.sum;
        if (it.b < K - 1)
            Q.emplace(Item(it.sum + B[it.b + 1] - B[it.b], it.b + 1));
    }
}

void read_array(int *p) {
    _for(i, 0, K) scanf("%d", &(p[i]));
    sort(p, p + K);
}

int main() {
    while (scanf("%d", &K) == 1) {
        read_array(A);
        _for(i, 1, K) read_array(B), merge();
        _for(i, 0, K) printf("%d%c", A[i], i < K - 1 ? ' ' : '\n');
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 3.1.2 节例题 4
注意：本题只用了两个数组便实现了 K 个数组的合并
同类题目：IOI 2020 Day 1, Carnival Tickets
*/

```

例4-11 【树的层次遍历】树的层次遍历（Trees on the level, Duke 1993, UVa 122）

输入一棵二叉树，你的任务是按从上到下、从左到右的顺序输出各个结点的值。每个结点都按照从根结点到它的移动序列给出（L表示左，R表示右）。在输入中，每个结点的左括号和右括号之间没有空格，相邻结点之间用一个空格隔开。每棵树的输入用一对空括号“()”结束（这对括号本身不代表一个结点），如图4-7所示。

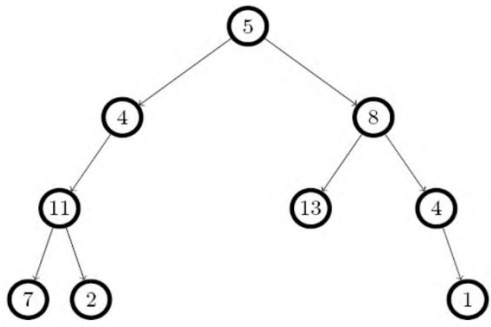


图4-7 一棵二叉树

注意：从根结点到某个叶子结点的路径上，如果有结点没有在输入中给出，或者给出超过一次，应当输出-1。结点个数不超过256。

【样例输入】

```
(11,LL) (7,LLL) (8,R) (5,) (4,L) (13,RL) (2,LLR) (1,RRR)
(4,RR) ()
(3,L) (4,R) ()
```

【样例输出】

```
5 4 8 11 13 4 7 2 1
-1
```

【代码实现】

```

// 陈锋
#include <bits/stdc++.h>
using namespace std;
#define _for(i, a, b) for (int i = (a); i < (b); ++i)
typedef long long LL;

struct Node {
    Node *left, *right;
    int val;
    bool hasValue;
    Node() : left(nullptr), right(nullptr), val(0), hasValue(false) {}
    ~Node() {
        if (left) delete left;
        if (right) delete right;
    }
};
typedef Node* PNode;
bool insert(PNode p, int val, const char* path) {
    assert(p);
    int len = strlen(path);
    _for(i, 0, len) {
        char c = path[i];
        if (c == 'L') {
            if (!p->left) p->left = new Node();
            p = p->left;
        } else if (c == 'R') {
            if (!p->right) p->right = new Node();
            p = p->right;
        }
    }
    p->val = val;
    if (p->hasValue) return false;
    return p->hasValue = true;
}

bool bfs(PNode p, vector<int>& ans) {

```

```

queue<PNode> Q;
ans.clear(), Q.push(p);
while (!Q.empty()) {
    p = Q.front(); Q.pop();
    if (!p->hasValue) return false;
    ans.push_back(p->val);
    if (p->left) Q.push(p->left);
    if (p->right) Q.push(p->right);
}
return true;
}

int main() {
    char s[512];
    int v;
    vector<int> ans;
    while (true) {
        Node nd, *root = &nd;
        bool valid = true;
        while (true) {
            if (scanf("%s", s) != 1) return 0;
            if (strcmp(s, "()") == 0) break;
            sscanf(s + 1, "%d", &v);
            if (!insert(root, v, strchr(s, ',') + 1)) valid = false;
        }
        if (valid && bfs(root, ans))
            for (size_t i = 0; i < ans.size(); i++)
                printf("%d%s", ans[i], (i == ans.size() - 1 ? "\n" : " "));
        else
            puts("not complete");
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典（第2版）》例题 6-7
同类题目：Cells, ACM/ICPC 杭州 2005, UVa 1357
*/

```

例4-12 【双向链表】移动盒子 (Boxes in a Line, UVa 12657)

你有一行盒子，从左到右依次编号为 $1, 2, 3, \dots, n$ 。可以执行以下4种操作指令。

- 1 $X\ Y$: 表示把盒子 X 移动到盒子 Y 左边（如果 X 已经在 Y 的左边，则忽略此指令）。

- ❑ 2 $X Y$: 表示把盒子 X 移动到盒子 Y 右边（如果 X 已经在 Y 的右边，则忽略此指令）。
- ❑ 3 $X Y$: 表示交换盒子 X 和 Y 的位置。
- ❑ 4: 表示反转整条链。

指令应保证合法，即 X 不等于 Y 。例如，当 $n=6$ 时，在初始状态下执行1 1 4操作后，盒子序列变为2, 3, 1, 4, 5, 6。接下来执行2 3 5，盒子序列变成2, 1, 4, 5, 3, 6。再执行3 1 6，得到2, 6, 4, 5, 3, 1。最终执行4，得到1, 3, 5, 4, 6, 2。

输入包含不超过10组数据，每组数据第一行为盒子个数 n 和指令条数 m ($1 \leq n, m \leq 100\,000$)，以下 m 行每行包含一条指令。每组数据输出一行，即所有奇数位置的盒子编号之和。位置从左到右编号为 $1 \sim n$ 。

【样例输入】

```
6 4
1 1 4
2 3 5
3 1 6
4
6 3
1 1 4
2 3 5
3 1 6
100000 1
4
```

【样例输出】

```
Case 1: 12
Case 2: 9
Case 3: 2500050000
```

【代码实现】

```

// 刘汝佳
#include <bits/stdc++.h>
using namespace std;
typedef long long LL;
const int NN = 1e5 + 4;
int Left[NN], Right[NN];
inline void link(int L, int R) { Right[L] = R; Left[R] = L;}
int main() {
    ios::sync_with_stdio(false), cin.tie(0);
    for (int n, m, kase = 1; cin >> n >> m; kase++) {
        for (int i = 1; i <= n; i++) Left[i] = i - 1, Right[i] = (i + 1) % (n + 1);
        Right[0] = 1, Left[0] = n;
        bool inv = false;
        for (int i = 0, op, X, Y; i < m; i++) {
            cin >> op;
            if (op == 4) inv = !inv;
            else {
                cin >> X >> Y;
                if (op == 3 && Right[Y] == X) swap(X, Y);
                if (op != 3 && inv) op = 3 - op;
                if (op == 1 && X == Left[Y]) continue;
                if (op == 2 && X == Right[Y]) continue;
                int LX = Left[X], RX = Right[X], LY = Left[Y], RY = Right[Y];
                if (op == 1) link(LX, RX), link(LY, X), link(X, Y);
                else if (op == 2) link(LX, RX), link(Y, X), link(X, RY);
                else if (op == 3) {
                    if (Right[X] == Y) link(LX, Y), link(Y, X), link(X, RY);
                    else link(LX, Y), link(Y, RX), link(LY, X), link(X, RY);
                }
            }
        }

        LL ans = 0;
        for (int i = 1, b = 0; i <= n; i++) {
            b = Right[b];
            if (i % 2 == 1) ans += b;
        }
        if (inv && n % 2 == 0) ans = (LL)n * (n + 1) / 2 - ans;
        printf("Case %d: %lld\n", kase, ans);
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典（第2版）》例题 6-5
注意：Link 函数是如何简化链表的各种操作的，以及翻转标志的使用
同类题目：Brute Force Sorting, HDU 6215
*/

```

例4-13 【递归下降法；表达式树】括号 (Brackets Removal, NEERC 2005, UVa 1662)

给出一个长度为 n 的表达式，包含字母、二元四则运算符和括号，要求去掉尽量多的括号。去括号规则如下：若 A 和 B 是表达式，则 $A+(B)$ 可变为 $A+B$ ， $A-(B)$ 可变为 $A-B'$ ，其中 B' 由 B 把顶层“+”与“-”互换得到；若 A 和 B 为乘法项 (term)，则 $A*(B)$ 变为 $A*B$ ， $A/(B)$ 变为 A/B' ，其中 B' 由 B 把顶层“*”与“/”互换得到。本题只能用结合律，不能用交换律和分配律。

例如， $((a-b)-(c-d)-(z*z*g/f)/(p*(t)*((y-u)))$ 去掉括号以后为 $a-b-c+d-z*z*g/f/p/t*(y-u)$ 。

【代码实现】

```
// 陈锋
#include <iostream>
#include <cassert>
#include <cctype>
using namespace std;
const int MAXN = 1024;

struct Node {
    char ch;
    Node *left, *right;
    bool enclose;           // 是否在括号内
    int priority;           // 优先级
    Node(char c = 0) : ch(c), left(NULL), right(NULL), enclose(false), priority(0)
    {
        if (ch == '*' || ch == '/') priority = 2;
        else if (ch == '+' || ch == '-') priority = 1;
    }
};
```

```

    }
    bool isOp() const { return !islower(ch); }
    ~Node() { // 析构函数，递归释放内存
        if (left) delete left;
        if (right) delete right;
    }
};

typedef Node* PNode;
string EX;
PNode pRoot;

ostream& operator<<(ostream& os, const PNode p) {
    if (!p) return os;
    if (p->enclose) os << '(';
    os << p->left << p->ch << p->right;
    if (p->enclose) os << ')';
    return os;
}

void reverse(PNode p) {
    assert(p);
    assert(p->isOp());
    char c = p->ch;
    switch (c) {
        case '+': p->ch = '-'; break;
        case '-': p->ch = '+'; break;
        case '*': p->ch = '/'; break;
        case '/': p->ch = '*'; break;
        default:
            assert(false);
    }

    PNode pl = p->left, pr = p->right;
    if (pl && pl->isOp() && pl->priority == p->priority) reverse(pl);
    if (pr && pr->isOp() && !pr->enclose && pr->priority == p->priority)
        reverse(pr);
}

void proc(PNode p) {
    assert(p);
    if (!p->isOp()) return;
    PNode pl = p->left, pr = p->right;
    if (pl && pl->isOp()) {
        if (pl->priority >= p->priority) pl->enclose = false;
        proc(pl);
    }

    if (pr && pr->isOp()) {
        if (pr->priority > p->priority) pr->enclose = false;
    }
}

```

```

    else if (pr->priority == p->priority) {
        if ((p->ch == '/' || p->ch == '-') && pr->enclose)
            pr->enclose = false, reverse(pr);
        pr->enclose = false;
    }
    proc(pr);
}
}

PNode parse(int l, int r) {
    assert(l <= r);
    char lc = EX[l];
    if (l == r) return new Node(lc);

    int p = 0, c1 = -1, c2 = -1;
    for (int i = l; i <= r; i++) {
        switch (EX[i]) {
            case '(': p++; break;
            case ')': p--; break;
            case '+': case '-': if (!p) c1 = i; break;
            case '*': case '/': if (!p) c2 = i; break;
        }
    }

    if (c1 < 0) c1 = c2;
    if (c1 < 0) {
        PNode ans = parse(l + 1, r - 1);
        ans->enclose = true;
        return ans;
    }

    PNode ans = new Node(EX[c1]);
    PNode ln = ans->left = parse(l, c1 - 1), rn = ans->right = parse(c1 + 1, r);
    assert(ans->priority);
    if (!ln->isOp()) ln->enclose = false;
    if (!rn->isOp()) rn->enclose = false;
    return ans;
}

int main() {
    while (cin >> EX) {
        pRoot = parse(0, EX.size() - 1);
        pRoot->enclose = false;
        proc(pRoot);
        cout << pRoot << endl;
        delete pRoot;
    }
    return 0;
}

```

```
/*
算法分析请参考：《算法竞赛入门经典——习题与解答》习题 11-6
注意：本题中如何使用析构函数来递归释放整棵树的内存，以及如何通过对“<<”操作符的重载实现整棵
树的递归输出
*/
```

例4-14 【并查集】易爆物 (X-Plosives, UVa 1160)

有一些简单化合物，每种化合物都是由两类元素组成的（每类元素用一个大写字母表示）。你是一个装箱工人，从实验员那里按照顺序依次把一些简单化合物装到车上。但这里存在一个安全隐患：如果车上存在 k 种简单化合物，正好包含 k 类元素，那么它们将组成一个易爆的混合物。为了安全起见，每当你拿到一种化合物时，如果判断它能和已装车的化合物形成易爆混合物，你就应当拒绝装车；否则就应该装车。编程输出有多少种没有装车的化合物。

输入包含多组数据。每组数据包含若干行，每行为两个不同的整数 a, b ($0 \leq a, b \leq 10^5$)，代表一个由元素 a 和元素 b 组成的简单化合物。所有简单化合物按照交给你的先后顺序排列。每组数据用一行-1结尾。输入结束标志为文件结束符 (EOF)。对于每组数据，输出没有装车的化合物的个数。

【代码实现】

```

// 陈锋
#include <bits/stdc++.h>
using namespace std;
typedef long long LL;
const int MAXN = 1e5 + 4;
int Pa[MAXN]; // 并查集
int findPa(int u) { return Pa[u] == u ? u : (Pa[u] = findPa(Pa[u])); }

int main() {
    while (true) {
        for(int i = 0; i <= MAXN; i++) Pa[i] = i;
        int ans = 0, u, v;
        while (true) {
            if (scanf("%d", &u) != 1) return 0;
            if (u == -1) break;
            scanf("%d", &v), u = findPa(u), v = findPa(v);
            if (u == v) ans++;
            else Pa[u] = v;
        }
        printf("%d\n", ans);
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 3.1.3 节例题 5
*/

```

例4-15 【带权并查集】合作网络（Corporative Network, POJ 1962）

有 n ($5 \leq n \leq 20\,000$) 个结点，初始时每个结点的父结点都不存在。你的任务是执行一次I操作和E操作，格式如下。

- ☐ I $u\ v$: 把结点 u 的父结点设为 v ，距离为 $|u-v|$ 除以1000的余数。输入应保证执行指令前 u 没有父结点。
- ☐ E u : 询问 u 到根结点的距离。

【代码实现】

```

// 刘汝佳
#include <cstdlib>
#include <iostream>
#include <string>
using namespace std;

const int NN = 20000 + 10;
int pa[NN], d[NN];

int findset(int x) { // 并查集查找与路径维护操作
    int &p = pa[x];
    if (p == x) return x;
    int r = findset(p);
    d[x] += d[p];
    return p = r;
}

int main() {
    int T;
    cin >> T;
    for (int t = 0, n, u, v; t < T; t++) {
        string cmd;
        cin >> n;
        for (int i = 1; i <= n; i++) pa[i] = i, d[i] = 0;
        while (cin >> cmd && cmd[0] != 'O') {
            if (cmd[0] == 'E') cin >> u, findset(u), cout << d[u] << endl;
            if (cmd[0] == 'I') cin >> u >> v, pa[u] = v, d[u] = abs(u - v) % 1000;
        }
    }
    return 0;
}
/*
算法分析请参考:《算法竞赛入门经典——训练指南》升级版 3.1.3 节例题 6
同类题目: Almost Union-Find, UVa 11987
          Islands, ACM/ICPC CERC 2009, UVa 1665
*/

```

例4-16 【图的连通块 (DFS)】油田 (Oil Deposits, UVa 572)

输入一个 m 行 n 列的字符矩阵,统计字符“@”可组成多少个八连块。如果两个字符“@”所在的格子相邻(横、竖或者对角线方向),就说它们属于同一个八连块。例如,图4-8中有两个八连块。


```

****@
*@**@
*@**@
@@@*@
@@**@

```

图4-8 八连块

【代码实现】

```

// 陈锋
#include <stdio.h>
#include <string.h>
const int NN = 100 + 4;
char pic[NN][NN];
int M, N, C[NN][NN];
#define _for(i, a, b) for (int i = (a); i < (b); ++i)

void dfs(int r, int c, int id) {
    if (r < 0 || r >= M || c < 0 || c >= N) return;
    if (C[r][c] > 0 || pic[r][c] != '@') return;
    C[r][c] = id;
    for (int dr = -1; dr <= 1; dr++)
        for (int dc = -1; dc <= 1; dc++)
            if (dr != 0 || dc != 0) dfs(r + dr, c + dc, id);
}

int main() {
    while (scanf("%d%d", &M, &N) == 2 && M && N) {
        _for(i, 0, M) scanf("%s", pic[i]);
        memset(C, 0, sizeof(C));
        int cnt = 0;
        _for(i, 0, M) _for(j, 0, N) if (C[i][j] == 0 && pic[i][j] == '@')
            dfs(i, j, ++cnt);
        printf("%d\n", cnt);
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典（第2版）》例题 6-12
注意：如何用简短的代码遍历每个点的 8 个邻居
同类题目：Ancient Messages, ACM/ICPC WF 2011, UVa 1103
*/

```